

Inspiring Book Quotes Application

“LiteraryLift” (final name still tbd)

Software Design Document (SDD)

Simran Dhillon  
300211810

COMP 155 – ON1

February 14<sup>th</sup>, 2024

## **Table of Contents**

### **1. INTRODUCTION**

- 1.1 Purpose
- 1.2 Scope
- 1.3 Overview
- 1.4 Reference Material
- 1.5 Definitions and Acronyms

### **2. SYSTEM OVERVIEW**

### **3. SYSTEM ARCHITECTURE**

- 3.1 Architectural Design
- 3.2 Decomposition Description
- 3.3 Design Rationale

### **4. DATA DESIGN**

- 4.1 Data Description
- 4.2 Data Dictionary

### **5. COMPONENT DESIGN**

### **6. HUMAN INTERFACE DESIGN**

- 6.1 Overview of User Interface
- 6.2 Screen Images
- 6.3 Screen Objects and Actions

### **7. REQUIREMENTS MATRIX**

### **8. APPENDICES**

# 1. INTRODUCTION

## 1.1 Purpose

The purpose of this software design document is to outline the architectural layout and system intricacies of the book quote application which provides motivational quotes to user to start their day with. This document provides a comprehensive breakdown of the software's design, functionality, and implementation, ensuring a unified understanding of the application's inner workings. The intended audience includes computer science professors, students of computer science, software developers, testers, stakeholders, and users.

## 1.2 Scope

This daily book quotes app aims to deliver daily inspirational book quotes to users as notifications whenever user feels like getting a motivational boost to start their day with. The application allows users to receive a notification of an inspirational quote once the app is opened and fetches quotes from a text file of quotes. The goals of the project are to provide users with a positive start to their day and to create a seamless user experience for receiving motivational quotes.

## 1.3 Overview

This document is structured to provide a detailed description of the design architecture, including system components, data flow, user interface design, and interactions between different modules. Each section addresses specific aspects of the software design, facilitating comprehension and aiding in the development process.

## 1.4 Reference Material

The following documents and resources were consulted in the preparation of this Software Design Document (SDD) for the Book Quote Application:

GfG. (2023, May 20). *Different ways of Reading a text file in Java*. GeeksforGeeks. <https://www.geeksforgeeks.org/different-ways-reading-text-file-java/>

*GUI programming - Java Programming Tutorial*. (n.d.). [https://www3.ntu.edu.sg/home/ehchua/programming/java/j4a\\_gui.html](https://www3.ntu.edu.sg/home/ehchua/programming/java/j4a_gui.html)

*Inspirational quotes at BrainyQuote*. (n.d.). BrainyQuote. <https://www.brainyquote.com/>

Mintype. (2022, November 7). *How to make GUI in JAVA | (simple & easy)* [Video]. YouTube. <https://www.youtube.com/watch?v=r5oasVxC8Ko>

NetBeans, A. (n.d.). *Introduction to GUI building*. <https://netbeans.apache.org/tutorial/main/kb/docs/java/gui-functionality/>

*Popular quotes*. (n.d.). <https://www.goodreads.com/quotes>

thenewboston. (2015, March 4). *JavaFX Java GUI Tutorial - 1 - Creating a basic Window* [Video]. YouTube. <https://www.youtube.com/watch?v=FLkOX4Eez6o>

thenewboston. (2015b, March 4). *JavaFX Java GUI Tutorial - 2 - Handle User Events* [Video]. YouTube. [https://www.youtube.com/watch?v=S\\_JN7zO12H4](https://www.youtube.com/watch?v=S_JN7zO12H4)

thenewboston. (2015c, March 6). *JavaFX Java GUI Tutorial - 5 - Creating Alert Boxes* [Video]. YouTube. <https://www.youtube.com/watch?v=SpL3EToqaXA>

*What is modular programming - Javatpoint.* (n.d.). [www.javatpoint.com](http://www.javatpoint.com).  
<https://www.javatpoint.com/what-is-modular-programming>

## 1.5 Definitions and Acronyms

- **SDD:** Software Design Document, a detailed blueprint outlining the design and specifications of a software application.
- **API:** Application Programming Interface, a set of protocols, tools, and definitions that allow different software applications to communicate with each other.
- **UI:** User Interface, the point of interaction between the user and the software application.
- **GUI:** Graphical User Interface, a type of user interface that allows users to interact with electronic devices through graphical icons and visual indicators.
- **Java:** A programming language commonly used for developing web, mobile, and enterprise applications.
- **Library:** A collection of pre-compiled routines, functions, and procedures that can be used by software developers to perform specific tasks or implement specific features.
- **TBD:** to be decided. Some parts of project might change as it develops further.

## 2. SYSTEM OVERVIEW

The book quote application is a Java program designed to provide users with a curated selection of inspirational quotes from various literary works and notable figures. With an interactive user interface, the application allows users to access a diverse range of quotes easily.

Functionality:

- **Random Quote Generation:** The core functionality of the application is to generate a random quote from a file of predefined quotes each time the application is launched, and this will be displayed to the user with the implementation of GUI.
- **User Interaction:** Users can interact with the application through a graphical user interface (GUI), which presents the randomly selected quote in a visually appealing manner.
- **Sources for the Inspirational Quotes:** The application will display quotes from reputable websites, literature, and renowned individuals, ensuring a diverse and inspiring collection so that whenever the user need inspiration, they will find some through the myriad of quotes available through the application.

Context:

The program serves as a digital repository of wisdom, motivation, and insight, catering to individuals seeking daily inspiration, motivation, or reflection. Whether used for personal enrichment, educational purposes, or as a source of encouragement, the application provides a seamless and accessible platform for accessing thought-provoking quotes.

Design:

- **User Interface:** The application will feature a user-friendly interface designed to enhance the user experience. With simple and straightforward design, users can easily engage with the application and discover new quotes effortlessly.
- **Randomization Algorithm:** The program will utilize a randomization algorithm to select quotes from the file of available quotes, ensuring originality with each interaction with the user.
- **Integration:** The program uses with external sources (various websites or books etc.) to retrieve and update its collection of quotes (which will be in a text file that can be easily updated with new quotes), providing users with a dynamic and expanding repository of inspiration.

Background Information:

This book quote program idea was conceived with the intention of using technology to promote positivity, motivation, and personal growth. Inspired by the timeless wisdom found in literature, speeches, and writings of influential figures, the aim is to deliver these insights to users in a convenient and accessible format. Through careful curation and thoughtful design, the book quote app strives to enrich the lives of its users by encouraging a daily habit of inspiring reflection.

### **3. SYSTEM ARCHITECTURE**

#### **3.1 Architectural Design**

The application follows a modular program structure, with distinct subsystems responsible for specific roles and functionalities. These subsystems collaborate and come together to achieve the complete functionality of the whole program, providing users with the enriching experience of accessing inspirational quotes easily.

High-Level Subsystems:

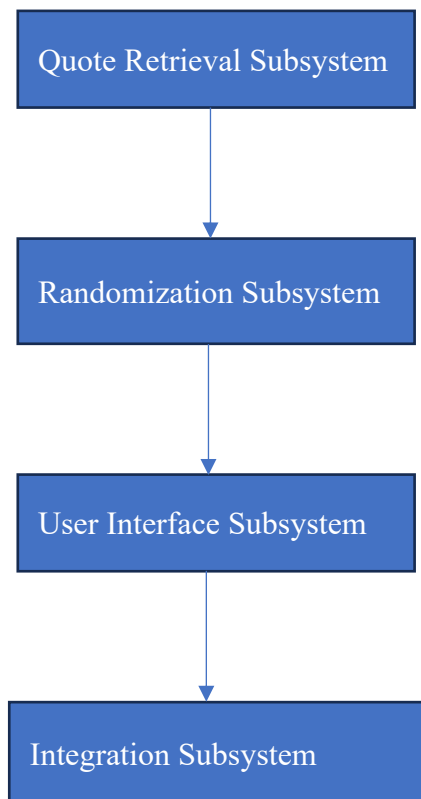
1. **Quote Retrieval Subsystem:**
  - **Responsibility:** This subsystem is responsible for retrieving quotes from a text file that contains various quotes from sources such as websites, literature, and renowned individuals.
2. **Randomization Subsystem:**
  - **Responsibility:** The Randomization Subsystem is tasked with selecting a random quote from the file of available quotes retrieved by the Quote Retrieval Subsystem so that user will always have a chance to learn from new quotes each day.
3. **User Interface Subsystem:**
  - **Responsibility:** This subsystem handles the presentation of the randomly selected quote to the user through a graphical user interface (GUI) which will be like a new window that shows on screen once the program is started by user.
4. **Integration Subsystem:**
  - **Responsibility:** The Integration Subsystem facilitates the integration of external sources (for example new quotes which programmer wants to update on text file) and updates the application's collection of quotes, ensuring a dynamic and diverse selection.

Collaboration:

- The Quote Retrieval Subsystem interacts with external sources such as websites and databases to fetch a wide array of quotes.
- The Randomization Subsystem receives the file of quotes from the Quote Retrieval Subsystem and selects a random quote to be displayed to user.
- The User Interface Subsystem presents the randomly selected quote to the user in a visually appealing manner, facilitating user interaction.
- The Integration Subsystem periodically updates the application's collection of quotes by integrating new quotes from external sources, ensuring a continuously enriching user experience.

Diagram:

This design ensures modularity, scalability, and maintainability of the book quote application, allowing for efficient collaboration between subsystems to deliver the desired functionality to the end-user.



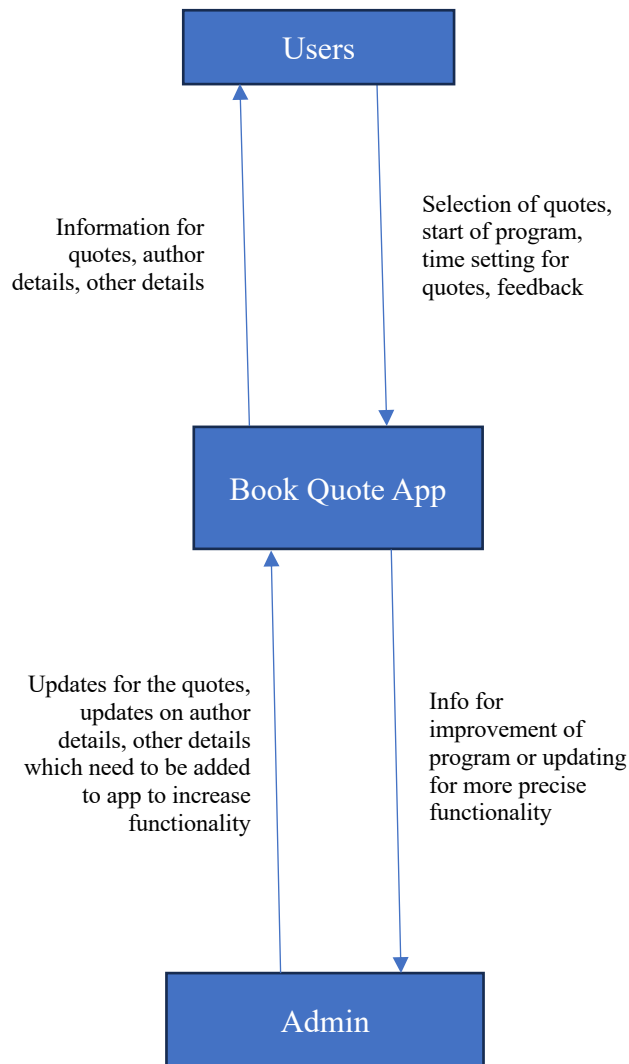
### 3.2 Decomposition Description

Functional Description:

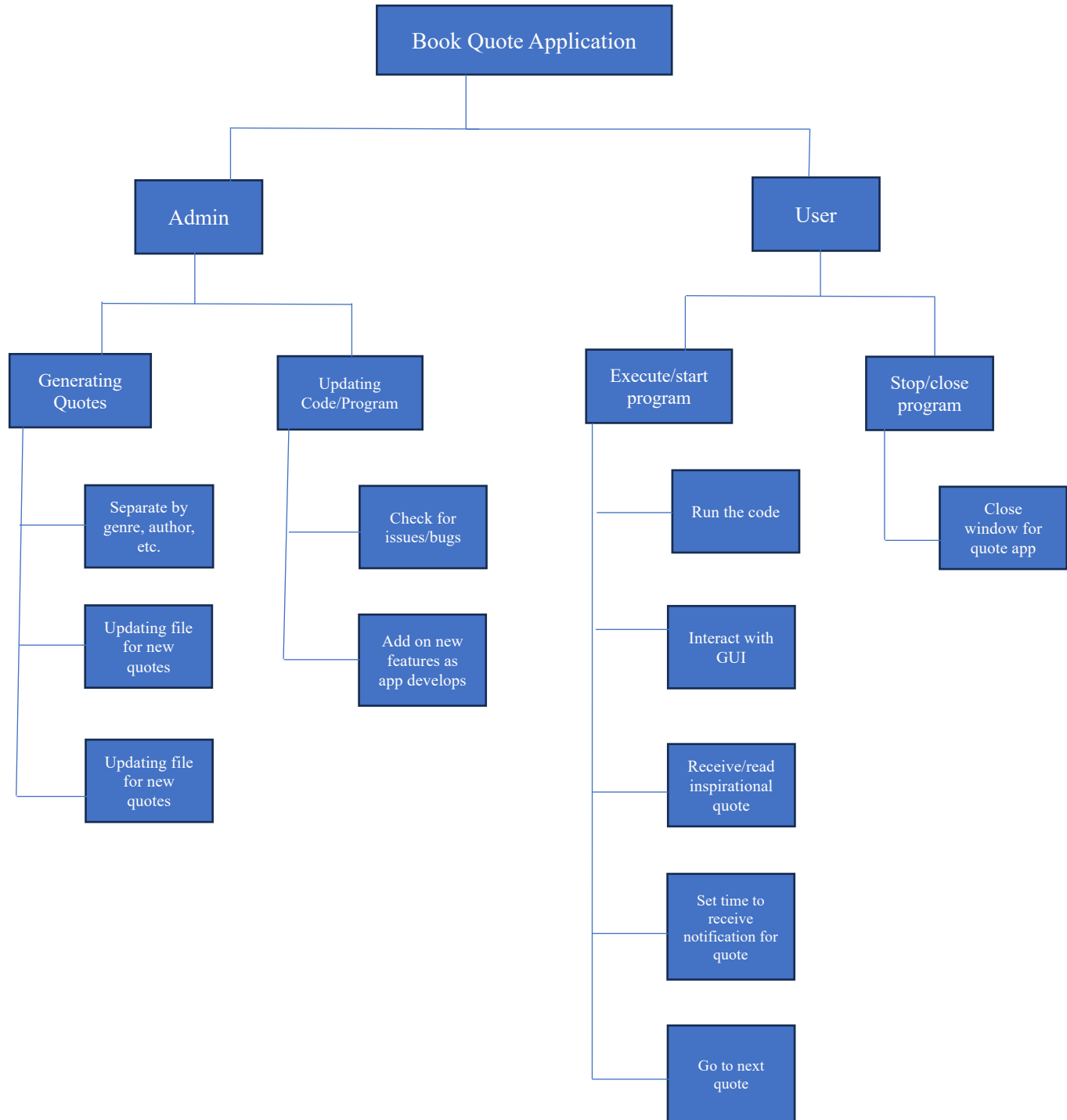
Top-Level Data Flow Diagram (DFD):

This diagram provides a high-level overview of the data flow within the Book Quote Application.

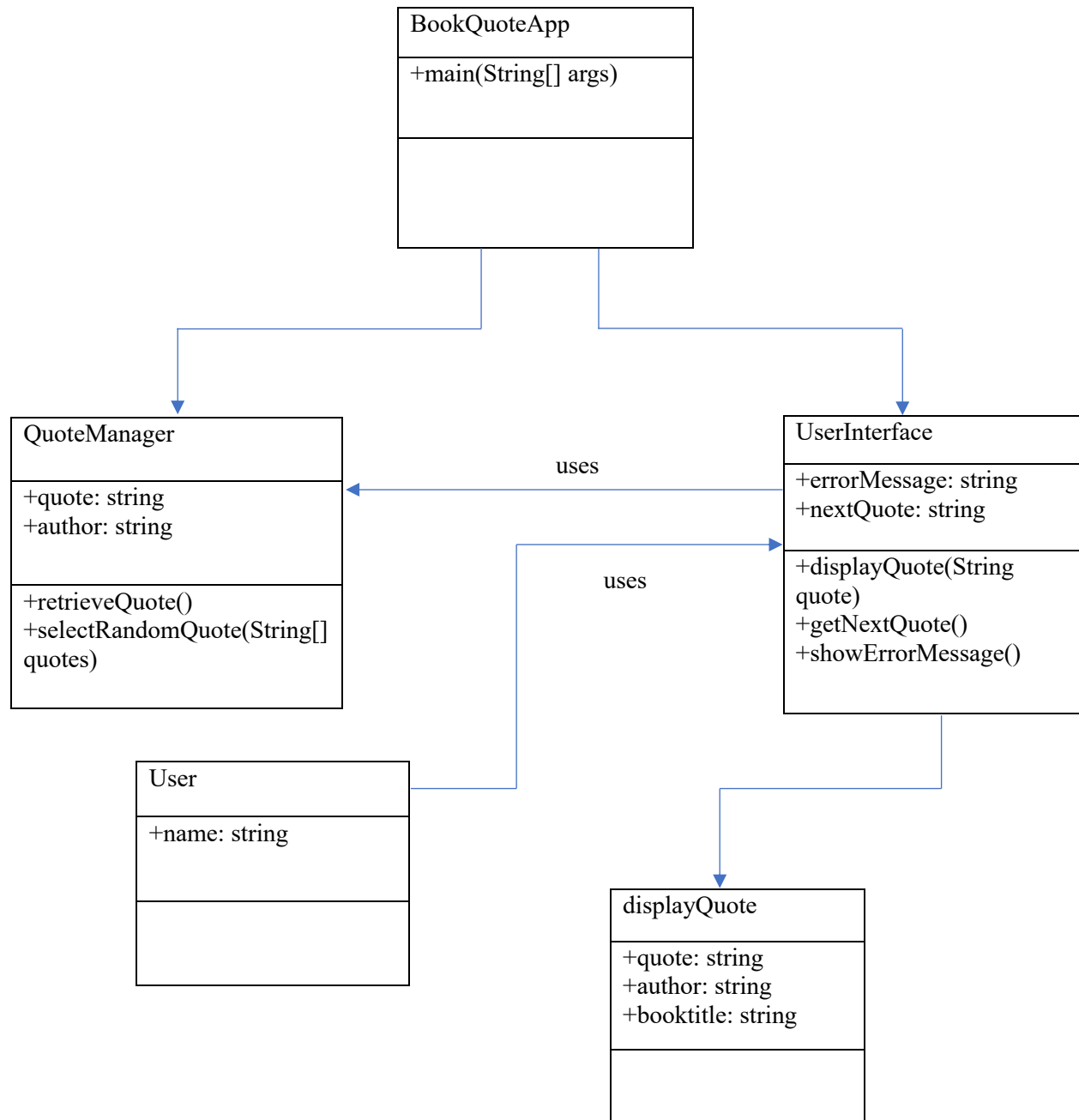
- Book Quote Application: Represents the main system.
- Users: interact with the main system by starting the program and clicking on the selection of quotes they want to see for the day. Further improvements of program can be to allow user to set time when they would like to see a quote pop up on screen at their desired time.
- Admin: the administration will then decide on improvements or updates for application and will also be responsible for adding in new quotes as well.



## Structural Decomposition Diagram:



UML Diagram:



### 3.3 Design Rationale

The selected architecture for the application was chosen after consideration of various factors, including scalability, maintainability, and user experience. The rationale for selecting this architecture is outlined below, along with critical issues and trade-offs that were considered.

A modular design approach was utilized, dividing the system into subsystems: Quote Retrieval, Randomization, and User Interface. This architecture was selected for these reasons:

- **Modularity:** By partitioning the main application system into distinct subsystems, each responsible for specific functionalities, there is better organization achieved. This modular design facilitates easier maintenance, debugging, and future enhancements of the program as needed when improvements arise.
- **Scalability:** The modular architecture allows for scalability, allowing addition of new features without significantly impacting existing components. This ensures that the application can accommodate future requirements and scale to meet increasing user concerns about things such as demand.
- **Code Reusability:** The design promotes code reusability by using functionalities within reusable components. For example, the Quote Retrieval subsystem can be reused in other projects or integrated with different applications that require quote retrieval capabilities as the quotes are generated in a file compiled from various sources.

Critical Issues and Trade-offs:

- **Complexity vs. Simplicity:** A more modular architecture introduces additional complexity, especially in terms of inter-subsystem coordination. However, this complexity is necessary to achieve a more maintainable and scalable system.
- **Programmer Expertise:** Considering the programmer's beginner status in coding, it was essential to avoid overly complex architectures that could pose significant challenges in development and maintenance. The chosen design contains a balance between providing a structured framework for learning and allowing for gradual skill development as well. By keeping the design approachable and intuitive, the programmer can focus on mastering fundamental programming concepts without feeling overwhelmed by unnecessary complexity.

In conclusion, the selected modular architecture offers a pragmatic approach to building the book quote application, addressing key concerns such as modularity, scalability, and code reusability. By carefully considering critical issues and trade-offs, a robust and maintainable solution that meets the needs of both developers and end-users can be developed.

## 4. DATA DESIGN

### 4.1 Data Description

The data design of the Book Quote Application involves transforming the information domain, consisting of quotes and relevant data, into structures suitable for storage, processing, and organization within the system. The major data entities include quotes themselves and any associated information such as author names or categories of book genres.

Data Structures:

- Quote Object: Each quote is represented as a structured object containing the quote text, author name, and possibly additional metadata such as category or source.

Storage and Organization:

- In-Memory Data Storage: Quotes and their associated metadata are stored in memory within the application. This allows for efficient retrieval and processing of quotes during runtime without the need for persistent storage.
- Data Processing: Quotes are processed and organized within the application to facilitate functionalities such as random quote generation and presentation to the user.

Databases or Data Storage Items:

- While the current implementation utilizes in-memory storage and uses a general text file to specifically store the databank of quotes, future enhancements of the application may incorporate a database for persistent storage of quotes, depending on the specific requirements and scalability needs of the application.

The data design of the Book Quote Application ensures efficient storage, retrieval, and processing of quotes, enabling seamless functionality and user interaction within the system.

### 4.2 Data Dictionary

System Entities:

1. File Path:
  - Type: String
  - Description: Represents the file path to the text file containing quotes. This path is used by the system to locate and read the quotes from the file.
2. Quote:
  - Type: String
  - Description: Represents a single quote extracted from the text file. Used to store individual quotes retrieved by the system.
3. Quote Array:
  - Type: Array of String
  - Description: An array data structure used to store multiple quotes retrieved from the text file. Provides a collection of quotes for further processing and display.
4. Quote File Reader:
  - Type: Class or Component
  - Description: A component responsible for reading quotes from the text file. It parses the file and extracts individual quotes for use by the application.
5. User Input:

- Type: String or User Interface Component
  - Description: Represents any input provided by the user through the graphical user interface. It includes commands, preferences, or interactions initiated by the user.
6. User Interface Component:
- Type: Class or Component
  - Description: Represents various graphical elements and components used in the user interface, such as buttons, text fields, and dialogs. These components facilitate user interaction with the application.

## 5. COMPONENT DESIGN

### Quote Retrieval Subsystem

Summary of Algorithm (Pseudocode):

- Function retrieveQuotes():
  - Initialize an empty list to store quotes.
  - For each source in list\_of\_sources:
  - Connect to the source and retrieve quotes.
  - Parse the retrieved data to extract quotes and associated metadata.
  - Add the parsed quotes to the list of quotes.
  - Return the list of retrieved quotes.

### Randomization Subsystem

Summary of Algorithm (Pseudocode):

- Function selectRandomQuote():
  - Generate a random index within the range of the number of quotes.
  - Retrieve the quote at the generated index from the list of quotes.
  - Return the randomly selected quote.

### User Interface Subsystem

Summary of Algorithm (Pseudocode):

- Function displayQuote():
  - Initialize the graphical user interface (GUI).
  - Display the quote text along with the author's name in a visually appealing manner.
  - Provide options for the user to interact with the displayed quote (e.g., share, save, generate another quote).

This component design section outlines the functionalities of each subsystem within the Book Quote Application, providing algorithms or object member functions in procedural description language or pseudocode.

## 6. HUMAN INTERFACE DESIGN

### 6.1 Overview of User Interface

The user interface of the application is designed to provide an agreeable experience for users to access and interact with inspirational quotes. From the user's perspective, the system allows for the following functionalities:

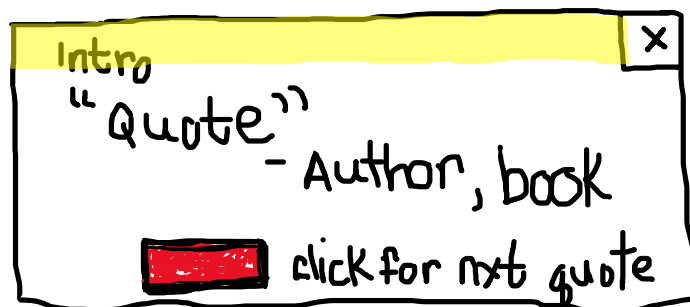
1. Quote Display:
  - Upon starting the program, users are presented with a randomly selected quote prominently displayed on the screen.
  - The quote is accompanied by the name of the author, providing context and attribution for the quote if there is any (since quotes will be compiled from various sources, not all will be from books).
2. Navigation and Interaction:
  - Users can navigate through the application using intuitive controls such as buttons or gestures.
  - Options are provided for users to interact with the displayed quote, such as generating another random quote with a single click.
3. Feedback Information:
  - The user interface provides feedback information to users to enhance their experience and provide guidance on interacting with the application.
  - Feedback may include instructions provided to user on the screen on what the program is about (full details on what will be in the final program display are still tbd).
  - Considering the chance for errors, an error messages should be displayed to inform users of any issues or limitations encountered during usage.

Overall, the user interface aims to be user-friendly, visually appealing, and informative, ensuring that users can easily access and engage with inspirational quotes to enrich their experience.

### 6.2 Screen Images

As the actual code is still under progress, a clear and concise description (along with a draft drawing) is given for what will be displayed for the user to see once they start the application or program.

1. Main Screen:
  - The main screen displays a quote prominently in the center, with the author's name displayed below it.
  - At the bottom of screen, buttons for users to interact with the quote such as "Next Quote".



2. Next Quote:
  - Tapping the "Next Quote" button generates and displays a new randomly selected quote, replacing the current one on the screen.
3. Intro
  - An introduction to the program and what it does will also be displayed to user so that they know what popped up on their window once they start the program is for their daily motivational quote application.
4. Close Window
  - Allows user to exit by choice once they have read their inspirational quote.

### 6.3 Screen Objects and Actions

The user interface consists of various screen objects, each associated with specific actions to facilitate user interaction. Here's a discussion of the key screen objects and their associated actions:

1. Quote Display:
  - Object Description: The central object on the screen, displaying the randomly selected quote.
  - Associated Actions: Users can view the quote text and the name of the author. No direct actions are associated with this object, as it serves as a passive display element.
2. Navigation Buttons:
  - Object Description: Buttons located at the bottom and top of the screen, allowing users to navigate through the application so that they may go to next quote or close application.
  - Associated Actions:
    - Next Quote: Generates and displays a new randomly selected quote, replacing the current one on the screen.
    - Close window which closes application

These screen objects and associated actions provide users with intuitive ways to interact with the application, allowing them to explore and engage with inspirational quotes effortlessly.

## 7. REQUIREMENTS MATRIX

This is a cross-reference that traces system components and data structures to the requirements outlined in the functional requirements, especially in all of section 3 above. The table below shows which system components satisfy each of the functional requirements.

| Functional Requirement                                       | Satisfied By                                            |
|--------------------------------------------------------------|---------------------------------------------------------|
| The system shall be able to retrieve quotes from a text file | QuoteRetriever Class, Quote File Reader Component       |
| The system shall be able to display quotes to the user       | GUIController Class, displayQuote() Function            |
| The system shall be able to select a random quote            | RandomQuoteSelector Class, selectRandomQuote() Function |
| The system shall provide a user interface for interaction    | GUIController Class, User Interface Subsystem           |

## 8. APPENDICES

More References for Project (after initial design, most of these later references are for the actual coding part as programmer needed lots of extra resources as a beginner in java.) :

Alex Lee. (2018, December 13). *Files in Java - Read text file easily #38* [Video]. YouTube. <https://www.youtube.com/watch?v=IHF1AYaNfdo>

Alex Lee. (2020, February 6). *Java GUI Tutorial - Make a GUI in 13 minutes #99* [Video]. YouTube. <https://www.youtube.com/watch?v=5o3fMLPY7qY>

Bro Code. (2020, September 14). *Java GUI: Full Course ☕ (FREE)* [Video]. YouTube. <https://www.youtube.com/watch?v=Kmg00avvEw>

GfG. (2023b, November 7). *Java JOptionPane*. GeeksforGeeks. <https://www.geeksforgeeks.org/java-joptionpane/>

Intellipaat. (2021, November 29). *Java Swing For Beginners | What is Java Swing | Java Swing Tutorial | Intellipaat* [Video]. YouTube. <https://www.youtube.com/watch?v=6zm8c6QFmjo>

*java for complete beginners - option panes*. (n.d.-b). [https://www.homeandlearn.co.uk/java/java\\_option\\_panes.html](https://www.homeandlearn.co.uk/java/java_option_panes.html)

*Java JFrame - javatpoint*. (n.d.). [www.javatpoint.com](http://www.javatpoint.com). <https://www.javatpoint.com/java-jframe>

*Java GUI frameworks. What to choose? Swing, SWT, AWT, SwingX, JGoodies, JavaFX, Apache Pivot?* (n.d.). Stack Overflow. <https://stackoverflow.com/questions/7358775/java-gui-frameworks-what-to-choose-swing-swt-awt-swingx-jgoodies-javafx>

McKenzie, C. (2022, September 16). User input with a Java JOptionPane example. *The Server Side*. <https://www.theserverside.com/blog/Coffee-Talk-Java-News-Stories-and-Opinions/Java-user-input-with-a-Swing-JOptionPane-example>

Muinuddin Muin. (2017, January 5). *Writing a text file in java using JFrame/GUI* [Video]. YouTube. <https://www.youtube.com/watch?v=y8IiDp5jgTc>

Pankaj. (2022, November 29). *How to Read a File Line-By-Line in Java*. DigitalOcean. <https://www.digitalocean.com/community/tutorials/java-read-file-line-by-line>

Ranjan-Beginner's Guide. (2022, October 4). *How to create GUI Applications in Java using Swing: Building Projects ( ComboBox, Password,jMenu)* [Video]. YouTube.  
<https://www.youtube.com/watch?v=gI4AYeqV3Xk>

Study Viral. (2017, December 11). *Java Swing Tutorials (GUI) - Read and Write JList with Text File - Study Viral* [Video]. YouTube. <https://www.youtube.com/watch?v=MIFj7neMatY>